

Software Development Fundamentals (SDF) – I

ODD 2023



Operators in C

Outline

- Assignment Operators
- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Misc Operators

Operators

- An operator are used to perform operation on the variables that yields a value.
- The variables, constants are used an operand with operator to form an expression.
- An operator is a symbol that tells the compiler that what operation needs to perform on the operands.
- Example : “ a + b “ : + is an **add operator** that adds two operands a and b.
- Some operators require two operands, while others act upon only one operand.

Expression

- An expression is formed using the operands such as constants, variables, and numbers with operators and parenthesis.

- Example :

$$(x * y) + z$$

In the example x , y and z are operands. * and + are operators.

C language provides the following types of operators:

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Misc Operators

Arithmetic operators

Arithmetic operators are used to perform arithmetical operation on the operand.

There are two types of arithmetic operation—

1. Unary arithmetic operators
2. Binary arithmetic operators

Unary arithmetic operators

- An unary operator needs single operands to perform the operation.
- Unary arithmetic operators are:
 - Unary minus (-)
 - Increment (+ +)
 - decrement (- -)
- Unary operators are **RIGHT** associative means they are **evaluated from right to left** when operators of same precedence are encountered in an expression.

Unary minus (-)

- Unary operator are written before any numeric value, variable or expression.

For example:

-y here '-' means change the sign of operand y.

- When unary operator are used on any operands, means it negates the value of operand.
- Following are few examples of unary operators:

-57

- (a*b)= it negates the value calculated by the operation a*b

-x = it negates the value of x. If x value is negative then it will become positive and if x value is positive then it becomes negative.

Increment operator (++)

- The increment (++) operator is a unary operator because it applies on a single operand.
- It adds **1** to its operand.
- increment operator represented as:
$$++\text{operand}/\text{operand}++$$
- **++operand** is called a prefix increment. In the Pre-Increment, value is first incremented and then used inside the expression.
- **operand++** is called as postfix increment. In the Post-Increment, value is first used in an expression and then incremented.

Example: `x = 4`

Prefix increment: `a = ++ x` : **x value first updated. a = 5, x=5**

Postfix increment: `a = x++` : **x value used directly then updated. a=4, x=5**

Example 1

```
int m=15, prem, postm;  
prem = ++m;  
postm= m++;  
Printf("\n m = %d, m);  
Printf("\n prem=%d, postm=%d,prem,postn);
```

Output:

m = 17

prem = 16, postm = 16

Exercise

Write the output of following:

```
void main()
{
    int a =3, b=6, c;
    c = (++a+b++);
    ++c;
    printf("a =%d, b= %d, c=%d",a,b,c);
}
```

Output?

a =4, b= 7, c=11

Decrement operator

- Decrement operator works same as increment operator.
- Decrement (- -) operator subtracts **1** from its operand.
- Decrement operator syntax:

--operands/operands--

- Postfix decrement (**operand- -**) : In this case value of operand is fetched before subtracting 1 from it.
- Prefix decrement (**- - operands**) : In this case value of operand is fetched after subtracting 1 from it.

Example 1

```
• int m=15, predm, postdm;  
• predm = --m;  
postdm= m--  
• Printf("\n m = %d, m);  
• Printf("\n predm=%d, postdm=%d,predm,postdm);
```

Output:

m = 13

predm = 14, postm = 14

Example

Write the output of following:

```
void main()
{
    int a =6, b=3, c;
    c = (--a-b--);
    c--;
    printf("a =%d, b= %d, c=%d",a,b,c);
}
```

Output?

a =5, b= 2, c=1

Binary arithmetic operators

- A binary operator performs arithmetic operation on the two operands.
- There are five binary operator.
 - + addition
 - subtraction
 - * multiplication
 - / division
 - % gives the remainder in integer division
- % (modulus operator) cannot be applied with floating point operands.
- There is no exponent operator in C. However there is a library function `pow( )` to carry out exponentiation operation.

Precedence of Arithmetic operators

Highest :

↑
++ --
- (unary minus)
*
/ %

Lowest

+ -

here ++ , -- are on same level and have higher precedence

%, /, * are one same level but precedence lower than -.

+ and - are on same level but precedence lower than %, /, *

Associativity

- Associativity is defined as the order in which consecutive operation with in the same precedence group will be carried out.
- Operators on same level of precedence are evaluated by the complier from left to right (Left associative).
- For a complex expression it becomes difficult to make out as to in what order the evaluation of sub expression would take place.
- In such cases we checkout the precedence and associativity.

E.g. in the expression

$$= 2+3-4$$

$$= 5-4$$

$$= 1$$

Exercise

```
int x= 30;  
printf ( "\n\t ++x=%d and x++ = %d",++ x, x++);
```

++x = 32 and x++ = 30

Precedence in Expression

$$1 + 2 * 3 - 4 / 5 = 1 + (2 * 3) - (4 / 5)$$



B.O.D.M.A.S

B stands for brackets
O for order (exponents)
D for division
M for multiplication
A for addition
S for subtraction

Example

```
int x;  
x= 7 + 3 * 5;
```

output:
x = 22

```
int x;  
x= ( 7 + 3 ) * 5;
```

output:
x = 50

```
int x;  
x= 7 / 3 * 5;
```

output:
x = 10

Modulus operator (%)

- This operator has same priority as that of multiplication and division
- $a \% b$ = output remainder after performing a / b
- Example:

$$7 \% 10 = 7$$

$$7 \% 1 = 0$$

$$7 \% 2 = 1$$

$$7 \% 7 = 0$$

Exercise

```
int y;  
y = 10 % 4 * 3;
```

Output:
6

```
int y;  
y = 3 * 10 % 4;
```

Output::
2

- $a * + b$: Invalid as two operators can not be used in continuation.
- $a(b * c)$: Invalid as there is no operator between a and b.

Important

- Modulus operator (%) : It produces remainder of an integer division. This operator **can not** be used with floating point numbers.

```
int main( ){  
    float  f_1=3.2, f_2=1.1, f_3 ;  
    f_3 = f_1 % f_2;  
    printf(" %f", f_3);  
    return 0; }
```

Output: **error at line 3**
illegal use of floating point

Example:

```
1  #include<stdio.h>
2  int main()
3  {
4  int a=5, b=3;
5  printf("Welcome to SDF-1 class\n\n");
6  printf("The Addition of %d and %d is: %d\n",a,b,a+b);
7  printf("The Subtraction of %d and %d is: %d\n",a,b,a-b);
8  printf("The Multiplication of %d and %d is: %d\n",a,b,a*b);
9  printf("The Division of %d and %d is: %d\n",a,b,a/b);
10 printf("The Modulus operation of %d and %d is: %d\n",a,b,a%b);
11 printf("The Incremented value ++a is: %d\n",++a);
12 printf("The Decrementd value --b is: %d\n",--b);
13 return 0;
14 }
15
```

• OUTPUT:

```
Welcome to SDF-1 class
```

```
The Addition of 5 and 3 is: 8
```

```
The Subtraction of 5 and 3 is: 2
```

```
The Multiplication of 5 and 3 is: 15
```

```
The Division of 5 and 3 is: 1
```

```
The Modulus operation of 5 and 3 is: 2
```

```
The Incremented value ++a is: 6
```

```
The Decrementd value --b is: 2
```

```
Process returned 0 (0x0)   execution time : 0.108 s
```

```
Press any key to continue.
```

Relational & Logical operators

- A relational operators are used to compare two operands.
- The result of such operation is always **logical** either **TRUE (1)** or **FALSE (0)**.

Following are the relational operators in C

	<	less than	$a < b$
❖	>	greater than	$a > b$
❖	<=	less than or equal to	$a <= b$
❖	>=	greater than or equal to	$a >= b$
❖	==	is equal to	$a == b$
❖	!=	is not equal to	$a != b$

Following table shows all the relational operators supported by C language. Assume variable **A** holds 10 and variable **B** holds 20, then:

Operator	Description	Example
==	Checks if the values of two operands are equal or not, if yes then condition becomes true.	A==B not true
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true	A!=B true
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	A>B is not true
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	A<B is true
<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	A <= B is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	A >= B is not true

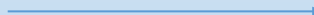
Exercise

- Suppose that a,b, and c are integers variables.

What will be the value of following expressions:

int a =10, b=5 , c =15;

a<b;  False

b<c;  True

(a+b)>=c;  True

(b+10)==(a+c);  False

a!=10;  False

Example:

```
#include <stdio.h>
int main()
{
    int a=5, b=5, c=10;
    printf("Welcome to SDF-1 class!\n\n");
    printf("For %d == %d : The output is: %d \n", a, b, a == b);
    printf("For %d == %d : The output is: %d \n", a, c, a == c);
    printf("For %d != %d : The output is: %d \n", a, c, a != c);
    printf("For %d != %d : The output is: %d \n", a, b, a != b);
    printf("For %d > %d : The output is: %d \n", a, b, a > b);
    printf("For %d > %d : The output is: %d \n", a, c, a > c);
    printf("For %d < %d : The output is: %d \n", a, b, a < b);
    printf("For %d < %d : The output is: %d \n", a, c, a < c);
    printf("For %d >= %d : The output is: %d \n", a, b, a >= b);
    printf("For %d >= %d : The output is: %d \n", a, c, a >= c);
    printf("For %d <= %d : The output is: %d \n", a, b, a <= b);
    printf("For %d <= %d : The output is: %d \n", a, c, a <= c);
    return 0;
}
```

• OUTPUT:

```
Welcome to SDF-1 class!
```

```
For 5 == 5 : The output is: 1
For 5 == 10 : The output is: 0
For 5 != 10 : The output is: 1
For 5 != 5 : The output is: 0
For 5 > 5 : The output is: 0
For 5 > 10 : The output is: 0
For 5 < 5 : The output is: 0
For 5 < 10 : The output is: 1
For 5 >= 5 : The output is: 1
For 5 >= 10 : The output is: 0
For 5 <= 5 : The output is: 1
For 5 <= 10 : The output is: 1
```

```
Process returned 0 (0x0)    execution time : 0.070 s
Press any key to continue.
```

Logical operator or Boolean Operator

- Logical operators are used to perform logical operations on the given expression.
- For combining expressions we use logical operators.
- These operators return 0 for false and 1 for true.
- The operands can be constants, variables or expressions.
- There are 3 logical operators in C language.

&&	Logical AND	a&& b
	Logical OR	a b
!	Logical NOT	!a

Logic AND

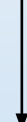
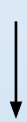
- ❖ The output of **AND** operation is **TRUE** if BOTH the operands are true.

Boolean Table

Condition1	Condition2	Result
False	False	False
False	True	False
True	False	False
True	True	True

Example 1

$(9 < 4) \ \&\& \ (6 > 7)$



0 && 0



False (0)

$(9 > 4) \ \&\& \ (2 < 3)$



1 && 1



True (1)

Logical OR

- The result of a *logical or* operation will be true if either operand is true or if both operands are true.

Boolean Table

Condition1	Condition2	Result
False	False	False
False	True	True
True	False	True
True	True	True

$(6 < 4) \mid \mid (3 > 5)$ is false

$(6 > 4) \mid \mid (3 > 5)$ is true

$(6 > 4) \mid \mid (3 < 5)$ is true

Logical NOT

- The Logical NOT (!) is a unary operator. It negates the value of the logical expression or operand.

Boolean Table

Condition	Result
True	False
False	true

- If value of $a = 0$ $!a = ?$
 $!a = 1$
- $!(3 < 5) \parallel (9 > 9) = ?$
 $!(1) \parallel (0) = !1 = 0 \rightarrow \text{false}$
- $!(34 \geq 765) = ??$
 $!0 \rightarrow 1 \rightarrow \text{True.}$

Conditional Operator

- Conditional operators required three operands or expression to execute the operation.
- That's why it is called as a ternary operator (? and:) .
- This written as:

TestExpression ? expression1 : expression2

- Firstly the TestExpression is evaluated.
- If TestExpression is true(nonzero), then expression1 is evaluated and it becomes the value of the overall conditional expression.
- If TestExpression is false(zero), then expression2 is evaluated and it becomes the value of overall conditional expression.

Example:

`a = 5, b = 8,`

`max = a > b ? a : b;`

In the expression `a > b` is evaluated, since it is false so the value of `b` becomes the value of conditional expression and it is assigned to variable `max`.

EX2: `a < b ? printf("a is smaller") : printf("b is smaller");`

Since the expression `a < b` is true, so the first `printf` function is executed.

Output: `a is smaller`

Comma Operator

- Comma operator (,) is a binary operator.
- When there are multiple expressions in a single line then that expressions will be separated by using comma operator.
- These expressions are evaluated one by one.
- The comma separated expressions are evaluated from left to right.
- The value of rightmost or last expression will be the value of the compound expression.

- Example:

$a = 8, b = 7, c = 9, a+b+c$

- Here we have combined 4 expressions. Initially 8 is assigned to the variable a, then 7 is assigned to the variable b, 9 is assigned to variable c and after this $a+b+c$ is evaluated which becomes the value of whole expression.
- So the value of the above expression is 24.

$\text{sum} = (a = 8, b = 7, c = 9, a+b+c);$

- Here the value of the whole expression on right side will be assigned to variable sum i.e. sum will be assigned value 24.
- Since precedence of comma operator is lower than that of assignment operator hence the parentheses are necessary here. The comma operator helps make the code more compact

sizeof Operator

- *sizeof* is an unary operator.
- This operator gives the size of its operand in terms of bytes.
- The operator can be a variable, constant or any datatype(int, float, char etc).
- For example sizeof(int) gives the bytes occupied by the int datatype i.e. 2

sizeof(float) = 4 byte

sizeof(char) = 1 byte

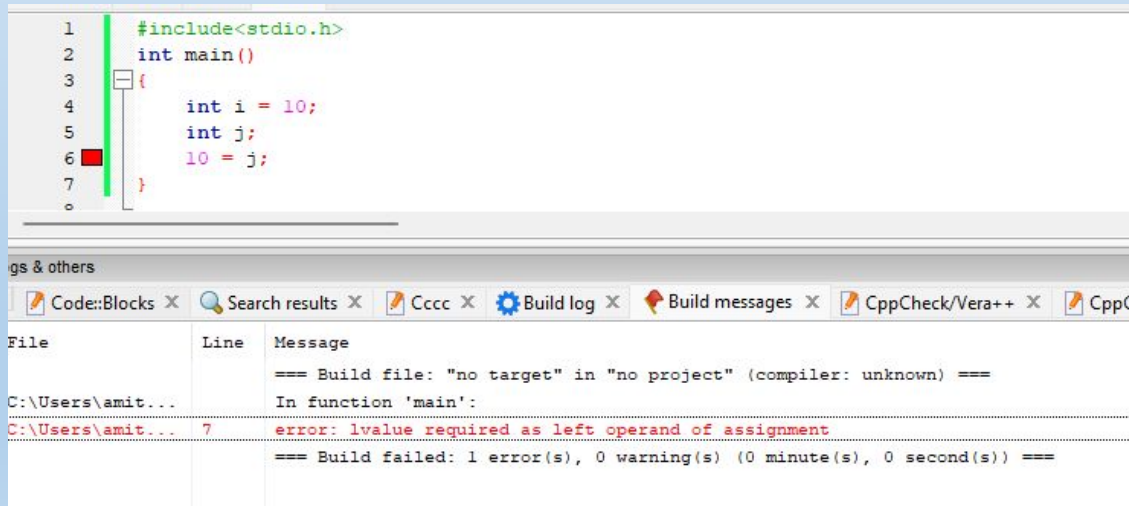
Operations using Assignment operators

Assignment operators allowed following operations in C:

- Assignment(=)
- Addition Assignment(+=)
- Subtraction Assignment(-=)
- Multiplication Assignment(*=)
- Division Assignment(/=)
- Remainder Assignment(%=)

Assignment operator

- It is used to **assign** a value to the variable:
`variable_name = expression;`
- **lvalue** : In compiler lvalue error messages means that an object on left hand side of assignment operator is missing.



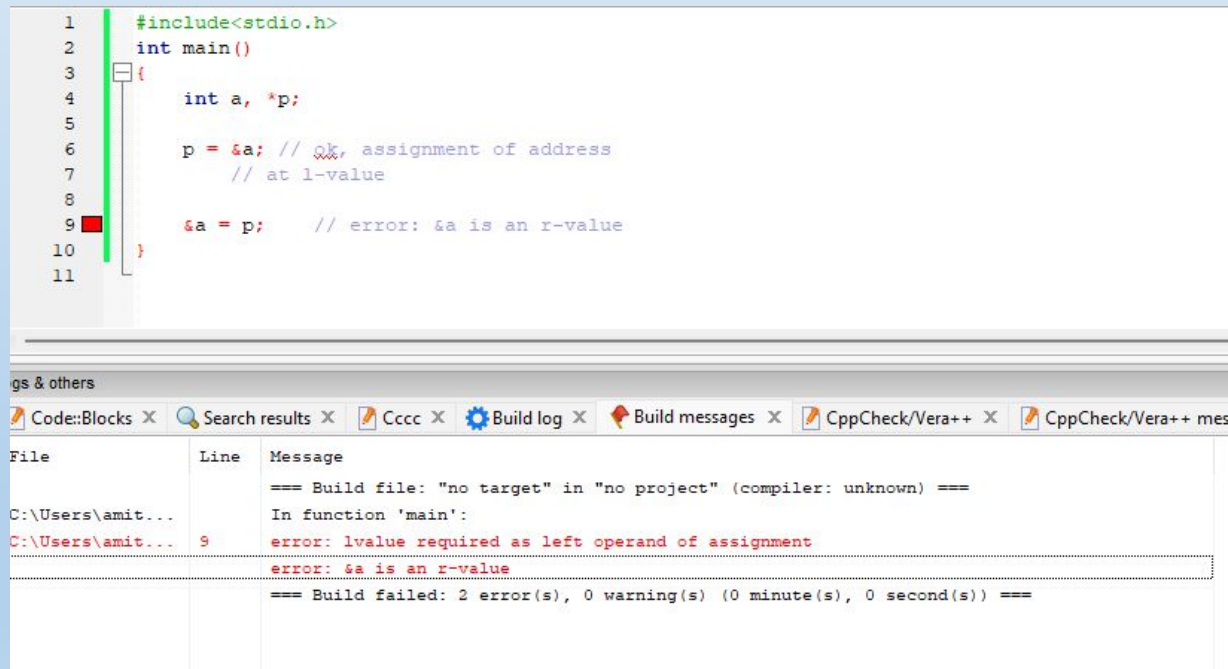
```
1  #include<stdio.h>
2  int main()
3  {
4      int i = 10;
5      int j;
6      10 = j;
7  }
```

The screenshot shows a code editor with a C++ program. Line 6 contains the statement `10 = j;`, which is highlighted with a red squiggly line. Below the code editor, a panel titled "Errors & others" displays the compiler's error message: "error: lvalue required as left operand of assignment". The message is shown in a table with columns for File, Line, and Message.

File	Line	Message
C:\Users\amit...	7	error: lvalue required as left operand of assignment

Assignment operator

- **rvalue** : In compiler rvalue error messages means that expression on right hand side of assignment operator is erroneous.



```
1  #include<stdio.h>
2  int main()
3  {
4      int a, *p;
5
6      p = &a; // Ok, assignment of address
7             // at l-value
8
9      &a = p; // error: &a is an r-value
10
11 }
```

The console window displays the following error messages:

```
=== Build file: "no target" in "no project" (compiler: unknown) ===
In function 'main':
C:\Users\amit... 9 error: lvalue required as left operand of assignment
error: &a is an r-value
=== Build failed: 2 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ===
```

Two cases of assignment

- Multiple assignment:

`int j=k=m=0;`

- Compound assignment:

`x= x+5;` this expression can be written as

`x += 5;`

similarly

`y= y-100;` is equivalent to `y -= 100;`

Example:

```
#include<stdio.h>
int main()
{
printf("Welcome to SDF-1 class!\n\n");
int number = 20, result;
result = number;
printf("result = %d \n", result);
result += number;
printf("result = %d \n", result);
result -= number;
printf("result = %d \n", result);
result *= number;
printf("result = %d \n", result);
result /= number;
printf("result = %d \n", result);
result %= number;
printf("result = %d \n", result);
return 0;
}
```

- **OUTPUT:**

```
Welcome to SDF-1 class!
```

```
result = 20
```

```
result = 40
```

```
result = 20
```

```
result = 400
```

```
result = 20
```

```
result = 0
```

```
Process returned 0 (0x0)   execution time : 0.065 s
```

```
Press any key to continue.
```

Bitwise Operators

- C has the ability to support the manipulation of data at the bit level.
- Bitwise operators are used for operations on individual bits.
- Bitwise operators operate on integers only. The bitwise operators are:

Operator	Meaning
&	bitwise AND
	bitwise OR
^	bitwise exclusive OR (XOR)
~	One's complement
<<	bitwise left shift
>>	bitwise right shift

Example:

```
#include <stdio.h>

int main()
{
    printf("Welcome to SDF-1 class!\n\n");
    int a = 20, b = 12;
    printf(" Bitwise AND operation %d & %d : %d\n", a, b, a&b);
    printf(" Bitwise OR operation %d | %d : %d\n", a, b, a|b);
    printf(" Bitwise XOR operation %d ^ %d : %d\n", a, b, a^b);
    printf(" Bitwise ONE'S COMPLEMENT ~ %d operation : %d\n", a, ~a);
    return 0;
}
```

• OUTPUT:

```
Welcome to SDF-1 class!
```

```
Bitwise AND operation 20 & 12 : 4
```

```
Bitwise OR operation 20 | 12 : 28
```

```
Bitwise XOR operation 20 ^ 12 : 24
```

```
Bitwise ONE'S COMPLEMENT ~ 20 operation : -21
```

```
Process returned 0 (0x0)    execution time : 0.064 s
```

```
Press any key to continue.
```

Left and Right Shift Operators

In general, if we shift a number by n position to left, the output will be $\text{number} * (2^n)$.

```
int main()  
{  
    a = 5, b = 9  
    printf("a<<1 = %d\n", (a << 1));  
    printf("b<<1 = %d", (b << 1));  
    return 0;  
}
```

OUTPUT:

```
a<<1 = 10  
b<<1 = 18
```

In general, if we shift a number by n position to right, the output will be $\text{number} / (2^n)$.

```
int main()  
{  
    a = 5, b = 9  
    printf("a<<1 = %d\n", (a << 1));  
    printf("b<<1 = %d", (b << 1));  
    return 0;  
}
```

OUTPUT:

```
a<<1 = 2  
b<<1 = 4
```

Operator precedence: It dictates the order of evaluation of operators in an expression.

Associativity: It defines the order in which operators of the same precedence are evaluated in an expression. Associativity can be either from left to right or right to left.

Source: <https://overiq.com/>

Operator	Description	Associativity
() [] . -> ++ --	Parentheses or function call Brackets or array subscript Dot or Member selection operator Arrow operator Postfix increment/decrement	left to right
++ -- + - ! ~ (type) * & sizeof	Prefix increment/decrement Unary plus and minus not operator and bitwise complement type cast Indirection or dereference operator Address of operator Determine size in bytes	right to left
* / %	Multiplication, division and modulus	left to right
+ -	Addition and subtraction	left to right
<< >>	Bitwise left shift and right shift	left to right
< <= > >=	relational less than/less than equal to relational greater than/greater than or equal to	left to right
== !=	Relational equal to and not equal to	left to right
&	Bitwise AND	left to right
^	Bitwise exclusive OR	left to right
	Bitwise inclusive OR	left to right
&&	Logical AND	left to right
	Logical OR	left to right
? :	Ternary operator	right to left
= += -= *= /= %= &= ^= = <<= >>=	Assignment operator Addition/subtraction assignment Multiplication/division assignment Modulus and bitwise assignment Bitwise exclusive/inclusive OR assignment	right to left
,	Comma operator	left to right

Exercise

```
#include<stdio.h>
main() {
    char x;
    int y;
    x=100;
    y=125;
    printf("%c\n",x);      d
    printf("%c\n",y);      }
    printf("%d\n",x);      100
    return 0;
}
```

```
#include<stdio.h>
#include<stdio.h>
main() {
    int x=100;
    printf("%d\n",10 + x++);    110
    printf("%d\n",10 + ++x);    112
    return 0;
}
```

```
#include <stdio.h>
int main()
{
    float c = 5.0;
    printf ("Temp in Fahrenheit is %.2f", (9/5)*c + 32);
    return 0;
}
```

Since 9 and 5 are integers, integer arithmetic happens in subexpression (9/5) and we get 1 as its value.
37.00 answer

- [https://www.tutorialspoint.com/cprogramming/c_relational_operators.htm\](https://www.tutorialspoint.com/cprogramming/c_relational_operators.htm)
- <https://www.programiz.com/c-programming/c-operators>

The End